Reusable Coding Agent Guidance

These instructions are designed to be shared across software projects. Adapt your decisions to the repository and the user's request; do not assume that a different project has this repository's product name, framework, tools, or deployment setup.

Understand the project first

- Read relevant source, tests, and existing documentation before making changes.
- Determine the project's real name, purpose, audience, and established terminology from reliable repository evidence such as user instructions, product documentation, UI copy, and existing brand assets. A package name, folder name, starter-template label, or inherited placeholder may not be the customer-facing product name.
- If the user asks to brand or rebrand a project, use the user's requested identity. Otherwise, when the intended identity is ambiguous or conflicting, ask before making widespread brand changes.
- Check `.agent/` when it exists. Read only the skills, workflows, and references relevant to the task. Treat it as optional supporting project guidance, not a required tool or a substitute for inspecting the code.
- Identify the actual stack, architecture, build commands, test commands, and deployment constraints from the repository. Do not copy assumptions or commands from another project.
- Follow established naming, formatting, architecture, and UI conventions. Prefer existing components and helpers over parallel implementations.

General working principles

Make focused, complete changes

- Address the root cause and the full requested behavior; avoid unrelated refactors and speculative features.
- Preserve existing behavior unless a change is intentional. Check related screens, APIs, and data flows so behavior stays consistent throughout the application.
- Keep error paths visible and useful. Do not silently discard failures or return success-shaped fallbacks for invalid input.
- Keep types accurate. Prefer explicit types and validation over unsafe casts.
- Update documentation and tests directly affected by a change.
- Never overwrite or discard unrelated user changes.
- Follow the user's latest explicit request when instructions conflict, while preserving security and data-safety requirements.

Protect users and their data

- Treat input crossing a trust boundary as untrusted; validate it at the appropriate boundary and enforce authorization on the server.
- Keep credentials, private data, and server-only configuration out of source control and browser-delivered code. Do not create or expose secrets as part of a fix.
- Do not weaken authentication, authorization, privacy, or production checks merely to make local development easier.
- Use only synthetic or explicitly authorized data for development and tests.
- Avoid destructive or irreversible actions unless requested and confirmed.

Build usable interfaces

- Preserve the product's visual language and interaction patterns.
- Design for keyboard use, assistive technology, readable contrast, clear focus, and appropriately sized touch targets.
- Check narrow screens as well as desktop layouts. Prevent clipped content, unintended horizontal scrolling, and controls obscured by overlays.
- Make loading, empty, success, and error states clear when they apply.

Verify and communicate

- Use the repository's existing tests, type checks, linters, and build scripts. Prefer focused checks first, then broaden them when the change warrants it.
- For UI changes, verify that the page renders and exercise important interactions when practical; a successful compile alone does not establish that a feature works.
- Report what changed, which checks passed, and any remaining failures or limitations. Distinguish newly introduced issues from existing warnings.
- Explain outcomes in product terms. Do not ask the user to run local commands or inspect implementation details when you can verify the result yourself.

Branding and product identity

When asked to identify, apply, or update a project's branding:

1. Establish the intended public-facing product name and terminology from the user's instructions or confirmed repository evidence. Ask if the evidence conflicts; do not infer a new brand from a technical package or folder name.
2. Audit the surfaces where identity is presented, as applicable: page titles, headers and navigation, login and onboarding, metadata and social previews, favicons, app or PWA manifests and icons, emails, generated documents, customer-facing copy, and project documentation.
3. Use supplied, approved brand assets where available. If essential assets are missing, ask for them or agree on an appropriate design before replacing them. Do not overwrite platform-, framework-, or vendor-owned assets.
4. Update references consistently, including asset paths and tests. Search for stale names, template labels, and obsolete asset paths after making changes.
5. Preserve technical identifiers, third-party notices, legal attribution, and platform integrations unless the user specifically authorizes an appropriate change. A customer-facing rebrand does not automatically mean removing required infrastructure.
6. Verify that updated assets load and that the key branded pages render correctly at desktop and mobile sizes.

Using `.agent/` guidance

`.agent/` is an optional project-level location for skills, workflows, and reference material:

- Confirm a referenced file exists before relying on it.
- Load only material relevant to the current task.
- Keep broadly reusable principles here; use focused `.agent/` material for detailed, specialized workflows when the project provides it.
- Resolve conflicting or stale instructions using the current user request and the actual repository behavior. Do not blindly apply instructions copied from another project.

Project-specific instructions

Before making a change, check for more specific repository guidance and configuration, including local `AGENTS.md` files, project documentation, dependency manifests, build configuration, and test setup. Follow applicable instructions, but verify that named files, commands, integrations, and constraints actually exist in the current project.